

Ontologia — Spécifications v1.0

Plateforme SaaS de gestion d'ontologies et de knowledge graphs d'entreprise, propulsée par Neo4j.

1. Vision produit

Proposition de valeur

Les équipes data et IA des entreprises construisent des knowledge graphs de plus en plus critiques — taxonomies produit, graphes d'entités métier, ontologies de conformité. Ces artefacts sont édités par plusieurs personnes, évoluent dans le temps, et doivent rester cohérents à travers les systèmes. Pourtant, il n'existe pas d'outil dédié qui combine la puissance d'un graphe natif (Neo4j) avec une expérience de gouvernance digne des outils modernes de développement logiciel.

Ontologia comble ce vide : c'est le "GitHub + Notion" des ontologies d'entreprise.

Positionnement

Dimension	Positionnement
Marché	Entreprises mid-market à large entreprise, secteurs data-intensifs (finance, santé, retail, industrie)
Modèle	SaaS multi-tenant, facturation par workspace/utilisateurs
Différenciation	Versioning git-like natif + workflows de gouvernance + Neo4j comme moteur de stockage
Concurrents indirects	Protégé (académique, lourd), TopBraid Composer (cher, complexe), Confluence+diagrammes (sans structure sémantique)

2. Personas utilisateurs

Persona A — "The Knowledge Architect" (utilisateur principal)

- Rôle** : Data architect, ontology engineer, knowledge engineer
- Profil** : 5–10 ans d'expérience, à l'aise avec les graphes, connaît peut-être OWL/SKOS

- **Besoins** : Modéliser des domaines complexes, maintenir la cohérence, gérer les évolutions dans le temps
- **Frustrations** : Les outils existants sont soit trop académiques, soit trop génériques (draw.io, Miro)
- **Usage Ontologia** : Crée et édite les ontologies, gère les branches, valide les pull requests conceptuelles

Persona B — "The Domain Expert" (contributeur)

- **Rôle** : Chef de produit, expert métier, data analyst
- **Profil** : Non-technicien, connaît son domaine mais pas le formalisme des graphes
- **Besoins** : Contribuer à la définition des concepts sans se perdre dans la syntaxe
- **Frustrations** : Dépend des architectes pour faire les moindres modifications
- **Usage Ontologia** : Suggère des modifications via l'interface visuelle, commente, approuve des changements

Persona C — "The Platform Engineer" (intégrateur)

- **Rôle** : Ingénieur data/ML, développeur backend
 - **Profil** : Consomme les ontologies dans des pipelines IA, RAG, systèmes de recherche
 - **Besoins** : API stable et versionnée, webhooks sur les changements, export dans différents formats
 - **Usage Ontologia** : Accède via API REST, configure des webhooks, consulte le changelog des versions
-

3. Fonctionnalités — Priorisation MoSCoW

Must Have (MVP)

M1 — Éditeur de graphe visuel Interface de modélisation interactive : créer/éditer des nœuds (concepts) et des arêtes (relations) avec labels et propriétés. Navigation dans le graphe avec zoom, pan, filtre par type.

M2 — Versioning git-like Chaque modification est enregistrée dans un commit atomique avec message, auteur et timestamp. Visualisation du diff entre deux versions (nœuds/relation ajoutés, modifiés, supprimés). Possibilité de revenir à une version antérieure.

M3 — Branches d'ontologie Créer des branches pour travailler sur une évolution sans affecter la version stable (main). Merge de branches avec détection de conflits au niveau conceptuel.

M4 — Multi-tenant & workspaces Isolation complète par organisation. Gestion des membres, rôles (Owner, Editor, Reviewer, Viewer). Plusieurs ontologies par workspace.

M5 — Import / Export Import depuis CSV (liste de concepts + relations), JSON-LD, OWL simplifié. Export vers JSON, CSV, JSON-LD.

M6 — Recherche & navigation Recherche full-text sur les concepts et leurs propriétés. Vue arborescente (hiérarchie is-a) en complément de la vue graphe.

Should Have (v1.1)

S1 — Workflows de révision (Pull Requests conceptuelles) Proposer un changement sur une branche → demande de review → approbation ou rejet avec commentaires → merge. Notifications email/in-app.

S2 — Commentaires & annotations Laisser des commentaires sur un concept, une relation ou un diff de version. Mentionner des collaborateurs.

S3 — API REST publique Endpoints pour lire/créer/modifier des concepts et relations. Authentification par API key. Documentation OpenAPI auto-générée.

S4 — Changelog & audit log Historique complet des actions par utilisateur. Export du changelog pour compliance.

S5 — Métadonnées enrichies Propriétés custom sur les concepts (synonymes, définition, source, statut de validation, score de confiance).

Could Have (v2.0)

C1 — Inférence simple Calculer les relations transitives (fermeture transitive de is-a), détecter les cycles, vérifier la cohérence.

C2 — Intégration IA Suggestion automatique de concepts manquants ou de relations via LLM (ex. "ce concept ressemble à X dans votre ontologie").

C3 — Webhooks & intégrations Webhooks sur les events (merge, nouveau commit). Connecteurs natifs vers dbt, Datahub, Atlan.

C4 — Query interface Cypher Console Cypher pour les utilisateurs avancés qui veulent requêter directement le graphe.

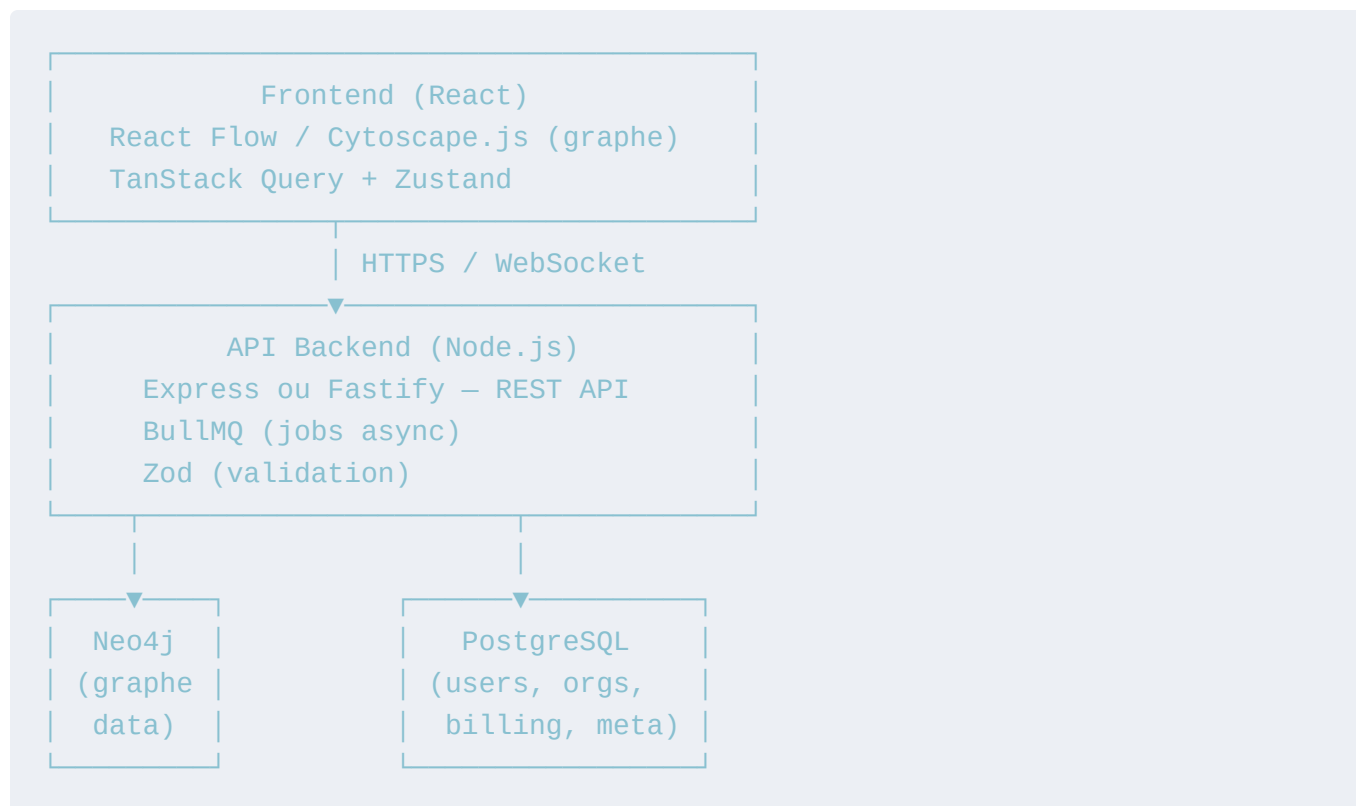
C5 — Templates d'ontologies Bibliothèque de templates métier (e-commerce, finance, santé) pour démarrer rapidement.

Won't Have (hors scope initial)

- Raisonnement OWL DL complet (trop complexe, trop niche)
- Éditeur de code OWL/Turtle natif (reste possible via import)
- Hébergement on-premise (possible plus tard en tier Enterprise)

4. Architecture technique

Stack recommandée



Pourquoi deux bases ? Neo4j stocke les ontologies (concepts, relations, versions). PostgreSQL gère tout ce qui est relationnel et transactionnel : comptes, organisations, abonnements, sessions, logs d'audit. Cette séparation évite d'abuser Neo4j pour des données qui ne bénéficient pas du modèle graphe.

Multi-tenancy

Approche recommandée : **un database Neo4j par organisation** (Neo4j Aura supporte ce pattern). Isolation forte, pas de risque de fuite de données entre tenants. Alternative moins chère : namespaces par propriété `orgId` sur tous les nœuds avec index dédié.

Stratégie de déploiement

- **Neo4j** : Neo4j Aura (managed cloud) — élimine la gestion d'infra dès le départ
- **Backend** : Render / Railway / Fly.io pour commencer, migration ECS/Kubernetes plus tard

- **Frontend** : Vercel (déploiement continu depuis main)
- **Auth** : Clerk ou Auth0 (déléguer l'authentification dès le MVP)

5. Modèle de données Neo4j

Méta-modèle

Le graphe Neo4j stocke deux couches : le **schéma** (quels types de concepts et relations existent) et les **instances** (les concepts concrets).

```
// — Ontologie & branches —————
(:Ontology {id, name, orgId, createdAt})
(:Branch   {id, name, ontologyId, createdAt, createdBy})

// — Versioning —————
(:Commit {
  id, branchId, message,
  authorId, timestamp,
  parentCommitId      // null pour le commit initial
})

// — Concepts (nœuds sémantiques) —————
(:Concept {
  id, commitId, branchId,
  name, description,
  status,                // draft | validated | deprecated
  properties: map        // propriétés custom (JSON sérialisé)
})

// — Relations typées —————
(:Concept)-[:RELATION {
  id, type,              // ex: "is-a", "part-of", "has-property"
  label,                 // label lisible
  commitId, branchId,
  properties: map
}]->(:Concept)

// — Types de relations (schéma) —————
(:RelationType {id, name, label, ontologyId, isTransitive, isSymmetric})
```

Stratégie de versioning (Event Sourcing léger)

Chaque modification génère un **Commit** atomique qui référence les concepts/relations dans leur état à ce moment. Un concept n'est jamais muté : on crée une nouvelle version du nœud liée au nouveau commit.

```
main:    C0 —> C1 —> C2 —> C3 (HEAD)
          |
feature/ C2' —> C3' (HEAD feature)
```

Le diff entre deux commits se calcule par comparaison des ensembles de concepts/rerelations à chaque commit. La vue "état actuel" d'une branche = tous les concepts au dernier commit de cette branche.

Avantage : historique immuable, rollback trivial (pointer HEAD vers un commit antérieur), diff natif.

6. Versioning & gouvernance — détail

C'est le cœur différenciant d'Ontologia. Voici les flows clés :

Flow 1 — Modification directe (Editor sur main)

```
Éditeur modifie un concept
→ UI crée un draft local (pas encore committé)
→ Éditeur clique "Commit changes"
→ Saisit un message ("Add definition to Product concept")
→ Commit créé dans Neo4j, HEAD de main avance
→ Changelog mis à jour, notifications envoyées
```

Flow 2 — Proposition de changement (Reviewer required)

```
Contributeur crée une branche "feature/add-product-variants"
→ Fait ses modifications sur cette branche
→ Ouvre une "Review Request" (équivalent PR)
→ Reviewers désignés reçoivent une notification
→ Peuvent commenter concept par concept sur le diff
→ Approuvent ou demandent des modifications
→ Merge dans main (fast-forward ou merge commit)
→ Branche archivée
```

Flow 3 — Rollback

```
Admin détecte une erreur dans la v3
→ Consulte l'historique des commits
```

→ "Revert to v2" → crée un nouveau commit qui restaure l'état de v2
→ Pas de suppression d'historique (audit trail intact)

Détection de conflits

Lors d'un merge, conflit si :

- Deux branches ont modifié le **même concept** (même id) différemment
- Une branche a supprimé un concept qu'une autre a modifié
- Résolution manuelle via UI de comparaison 3-way (ancêtre commun / branche A / branche B)

7. Design UI/UX — Concepts & Inspirations

Philosophie visuelle

L'interface doit transmettre **rigueur et clarté** sans être intimidante. Pas de look "logiciel académique des années 2000". S'inspirer de Linear (data-dense mais propre), Figma (collaboration fluide), et GitHub (versioning familier).

Palette de couleurs suggérée

Primaire : #6366F1 (indigo – intellectuel, tech, moderne)
Secondaire : #10B981 (émeraude – validation, success)
Accent : #F59E0B (ambre – warnings, diffs modifiés)
Danger : #EF4444 (rouge – suppressions, conflits)
Fond dark : #0F0F17 (quasi-noir bleuté)
Fond light : #F8F9FC (blanc légèrement bleuté)
Surface : #1C1C2E / #FFFFFF
Texte : #E2E8F0 (dark) / #1E293B (light)

Layout principal (3 zones)

Sidebar	Canvas (graphe)	Inspector
- Ontol.	[Visualisation interactive]	- Propriétés
- Branch	Nœuds colorés par type	- Historique
- Search	Arêtes avec labels	- Commentair.
- History	Mini-map en bas à droite	- Relations

Status bar : branche courante • nb commits • nb concepts

Vues principales

Vue Graphe (défaut) Canvas interactif avec React Flow. Les concepts sont des nœuds avec un chip coloré indiquant leur type. Les relations sont des arêtes orientées avec un label. Sélectionner un nœud ouvre l'Inspector à droite. Double-clic pour éditer inline.

Vue Arbre Représentation hiérarchique (is-a uniquement) en arbre collapsible. Idéale pour les taxonomies. Permet de drag-and-drop pour modifier les relations de hiérarchie.

Vue Diff Écran de comparaison entre deux commits ou deux branches. Code color : vert pour ajouts, rouge pour suppressions, jaune pour modifications. Peut être affiché en mode "graphe" ou en mode "liste".

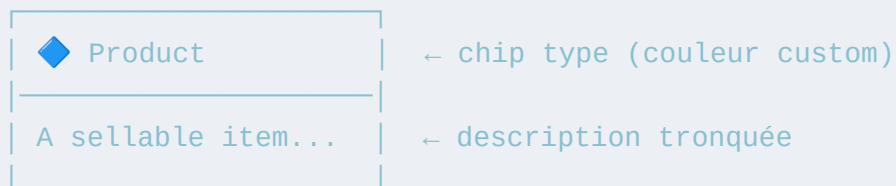
Vue Historique Timeline verticale des commits à la Git. Chaque commit montre l'auteur, le message, et le nombre de changements. Branches visualisées comme dans gitk/GitLens.

Conventions visuelles pour les concepts

Type de changement	Couleur dans le diff
Concept ajouté	Vert <code>#10B981</code> avec bordure
Concept supprimé	Rouge <code>#EF4444</code> avec strikethrough
Concept modifié	Ambre <code>#F59E0B</code>
Relation ajoutée	Ligne verte pointillée
Relation supprimée	Ligne rouge pointillée
Inchangé	Gris atténué

Éléments UI clés

Concept Node (dans le canvas) :




```
| [validated] [3 rel] | ← badge statut + nb relations
```

Commit panel (drawer bas ou modal) :

Commit changes

2 concepts modifiés · 1 relation ajoutée

Message : [_____]

Branch : • main

[Annuler]

 Commit

8. Roadmap produit

Phase 0 — Fondations (Mois 1-2)

- Setup infra (Neo4j Aura, auth, multi-tenant)
- Modèle de données + API CRUD concepts/relations
- UI canvas basique (React Flow) + Inspector

Phase 1 — MVP (Mois 3-4)

- Versioning (commits, historique, revert)
- Branches (create, switch, merge simple)
- Import CSV + Export JSON
- Recherche basique

Phase 2 — Gouvernance (Mois 5-6)

- Review Requests (PR conceptuelles)
- Commentaires et notifications
- Diff visuel entre versions/branches
- API REST publique + documentation

Phase 3 — Croissance (Mois 7-9)

- Détection de conflits avancée
- Webhooks & intégrations (dbt, Datahub)
- Suggestions IA (concepts similaires, relations manquantes)
- Templates métier

Phase 4 — Enterprise (Mois 10+)

- SSO (SAML/OIDC)
- Audit log exportable
- SLA, support dédié
- Option on-premise (Neo4j self-hosted)

9. Modèle SaaS & monétisation

Plan	Prix indicatif	Limites
Free	0 €	1 ontologie, 3 users, 500 concepts, pas de branches
Starter	~49 €/mois	5 ontologies, 10 users, illimité concepts, branches
Pro	~149 €/mois	Ontologies illimitées, 30 users, API, webhooks, audit log
Enterprise	Sur devis	Users illimités, SSO, SLA, support dédié, option on-prem

10. Questions ouvertes à trancher

1. **Nom du produit** — "Ontologia" est descriptif, mais peut manquer de caractère. Alternatives : *Konceptual*, *Graphia*, *OntoCraft*, *Semex*.
2. **Granularité du versioning** — Commit par action individuelle (comme un keylogger) ou commit manuel (comme Git) ? La proposition ci-dessus recommande le commit **manuel** pour que l'historique reste lisible.
3. **Gestion des suppressions** — Soft-delete (concept marqué "deprecated", reste visible dans l'historique) ou hard-delete (impossible, seulement archivage) ? Recommandation : **soft-delete uniquement** pour préserver l'audit trail.
4. **Schéma des types de relations** — Doit-on imposer que les types de relations soient déclarés à l'avance (schéma-first) ou permettre des relations libres (schéma-flexible) ? Recommandation : **flexible par défaut** avec possibilité de déclarer un schéma strict optionnel.

5. **Collaboration temps réel** — Curseurs live (Figma-style) dès le MVP ou différé à v2 ?
Recommandation : **différer**, implémenter d'abord le merge/diff proprement.
-

Document rédigé le 17 avril 2026 — Version 1.0